

MongoDB

B. Vijay Kumar

MongoDB : Content

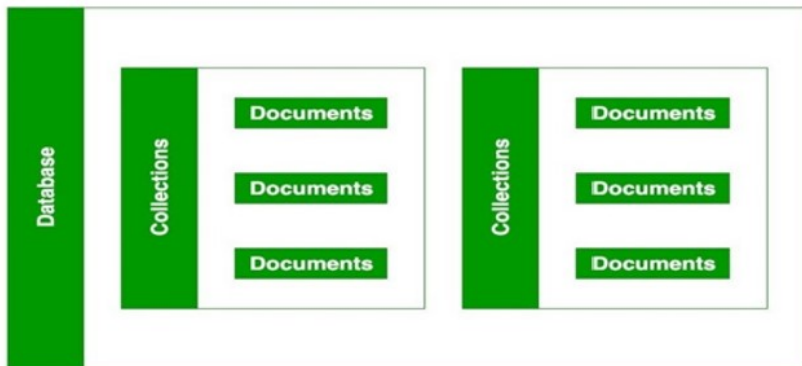
- What is MongoDB?
- Installing MongoDB?
- Collections and Documents
- Using MongoDB Compass
- Using MongoDB Shell
- Creating Database - Mongo Shell
- Adding New documents
- Finding Documents
- Sorting and Limiting Data
- Nested Documents
- Operators
- Updating documents
- Deleting documents

MongoDB : Content

- Connecting MongoDB using Node.js
- Reading Data from MongoDB
- Writing Data into MongoDB
- Questionnaire

What is MongoDB?

- MongoDB is an open-source document-oriented database.
- It is categorized under the NoSQL(Not only SQL) database because the storage and retrieval of data in the MongoDB are not in the form of tables.
- The data is stored in the collections and documents.



Features of MongoDB:

- Schema-less Database
- Document Oriented
- Indexing
- Replication
- Aggregation
- High Performance
- Scalability

How data stored in MongoDB:

In MongoDB, the basic unit of data is a document. A document is essentially a JSON-like structure composed of field-value pairs. MongoDB uses a binary-encoded format called BSON (Binary JSON) to store these documents.

Uses of MongoDB:

MongoDB is a popular NoSQL database known for its flexibility, scalability, and performance. It is widely used in various applications across different industries. Such as:

- Content Management System
- E-Commerce Platforms
- Indexing
- Real-Time Analytics
- Internet of things(IOT)
- Gaming Applications
- Customer Relation Management(CRM)
- Social Networks
- Big Data Applications
- Health Care
- Log Management

Advantages/Disadvantages of MongoDB:

Advantages:

- It is a schema-less NoSQL database. You need not to design the schema of the database when you are working with MongoDB.
- It provides great flexibility to the fields in the documents.
- It contains heterogeneous data.
- It provides high performance, scalability.
- It is a document oriented database and the data is stored in BSON documents.

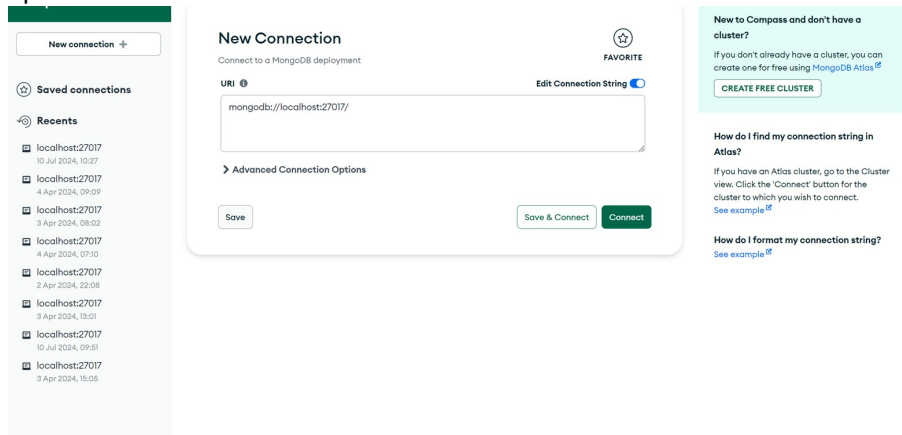
Disadvantages:

- It uses high memory for data storage.
- We are not allowed to store more than 16MB data in the documents.
- The nesting of data in BSON is also limited you are not allowed to nest data more than 100 levels.

Installing MongoDB

MongoDB Community Server:(Gives us MongoDB and Compass(GUI)) Download Link:

<https://www.mongodb.com/try/download/community-kubernetes-operator>



New connection +

Saved connections

- localhost:27017
10 Jul 2024, 10:27
- localhost:27017
4 Apr 2024, 09:09
- localhost:27017
3 Apr 2024, 08:02
- localhost:27017
4 Apr 2024, 07:10
- localhost:27017
2 Apr 2024, 22:08
- localhost:27017
3 Apr 2024, 13:01
- localhost:27017
10 Jul 2024, 09:51
- localhost:27017
3 Apr 2024, 15:05

New Connection

Connect to a MongoDB deployment

URI ⓘ Edit Connection String 

mongodb://localhost:27017/

Advanced Connection Options

Save Save & Connect Connect

FAVORITE

New to Compass and don't have a cluster?

If you don't already have a cluster, you can create one for free using [MongoDB Atlas](#)

[CREATE FREE CLUSTER](#)

How do I find my connection string in Atlas?

If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.

[See example](#)

How do I format my connection string?

[See example](#)

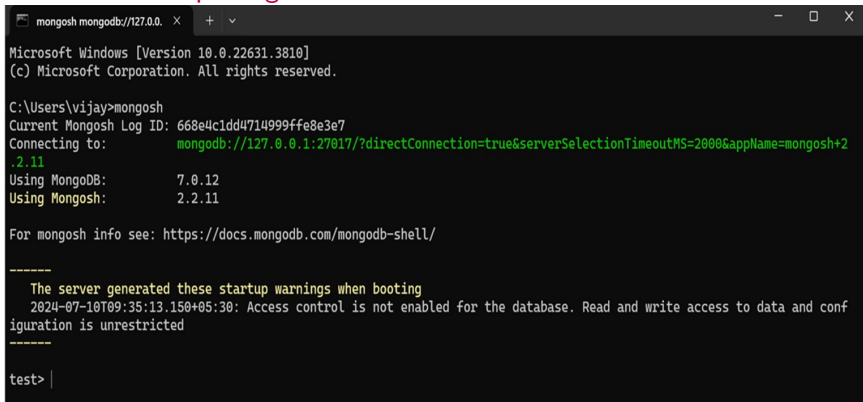
Installing MongoDB

MongoDB Shell:(CLI)

Download Link:

<https://www.mongodb.com/try/download/shell>

Note: Download package msi



```
mongosh mongodb://127.0.0.1 x + v
Microsoft Windows [Version 10.0.22631.3810]
(c) Microsoft Corporation. All rights reserved.

C:\Users\vijay>mongosh
Current Mongosh Log ID: 668e4c1dd4714999ffe8e3e7
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.2.11
Using MongoDB:      7.0.12
Using Mongosh:       2.2.11

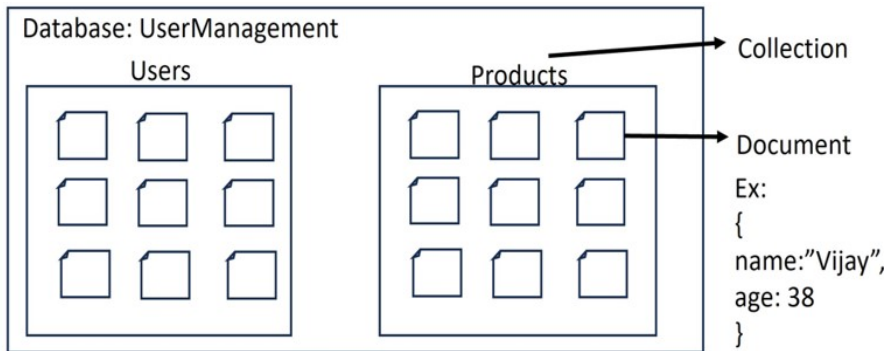
For mongosh info see: https://docs.mongodb.com/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2024-07-10T09:35:13.150+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> |
```

Collections and Documents

MongoDB Database Representation:

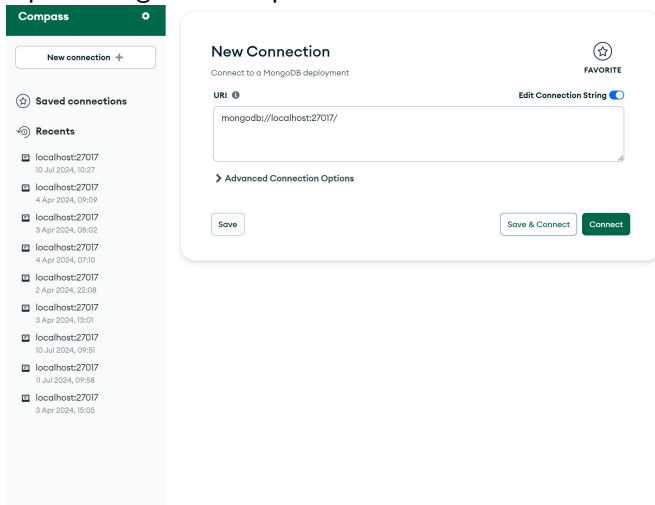


Collections and Documents

- **Database:** In MongoDB, A database is an organized collection of structured or unstructured information stored electronically on a machine locally or in the cloud.
- **Collections:** A collection is a grouping of MongoDB documents. Documents within a collection can have different fields.
- **Documents:** A document in MongoDB is a basic unit of data that is stored in JSON-like format (BSON – Binary JSON). Documents contain key-value pairs and can have nested documents and arrays.

Using MongoDB Compass

• Open MongoDB Compass From Startmenu



New to Compass and don't have a cluster?

If you don't already have a cluster, you can create one for free using [MongoDB Atlas](#)

[CREATE FREE CLUSTER](#)

How do I find my connection string in Atlas?

If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.

[See example](#)

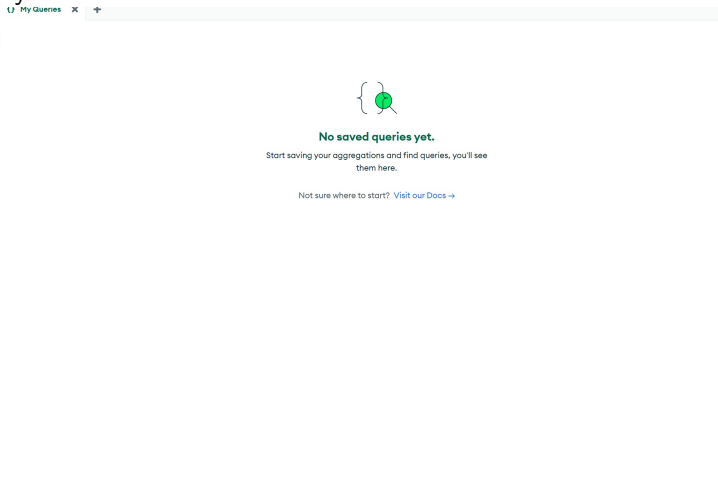
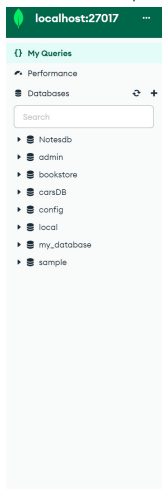
How do I format my connection string?

[See example](#)

• Click on Connect to Interact with MongoDB Server

Using MongoDB Compass

- Click on '+' Symbol on Left menu to create a database



Using MongoDB Compass

Create Database

Database Name

Collection Name

☐ **Time-Series**

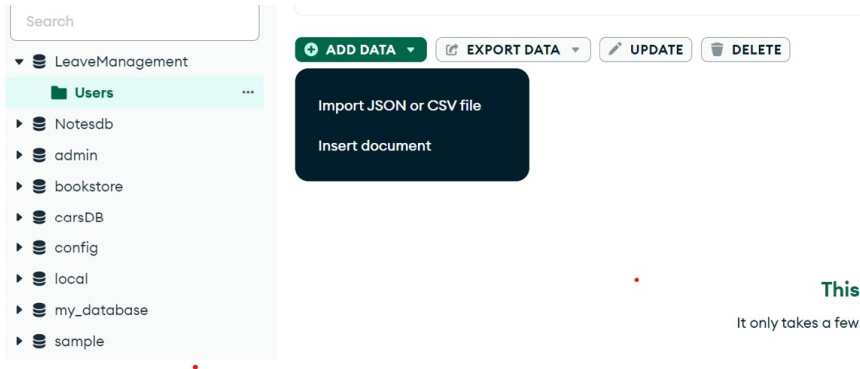
Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

➤ **Additional preferences** (e.g. Custom collation, Capped, Clustered collections)

CancelCreate Database

Using MongoDB Compass

- Select the collection of the database from the left menu
- Click on Add Data to add a new document/documents



We can add a new document by selecting Insert Document or
We can import the JSON or CSV file to insert the documents

Using MongoDB Compass

Default screen when we click on Insert Document

To collection LeaveManagement.Users

VIEW  

1 /**
2 * Paste one or more documents here
3 */
4 {
5 "_id": {
6 "\$oid": "669e5773e462014a350ebbf5"
7 }
8 }

Cancel Insert

We can add a Single document or we can add multiple documents

Using MongoDB Compass

Example: Adding Single Document



An Id will inserted automatically when we insert a document into the collection See below:



```
_id: ObjectId('669e5a17e462014a350ebbf8')
name: "Vijay"
city: "Hyd"
```

Using MongoDB Compass

We can insert multiple documents, multiple documents can be added as an array by selecting Insert Document from Add Data Menu check below:

Insert Document

To collection LeaveManagement.Users

VIEW



```
1  /**
2  * Paste one or more documents here
3  */
4  [
5  {
6    "name": "Ajay",
7    "city": "Hyd"
8  },
9  {
10   "name": "Shiva",
11   "city": "Mumbai"
12 },
13 ]
14
```

Cancel

Insert

Using MongoDB Compass

User collection after inserting 2 documents



```
_id: ObjectId('669e5a17e462014a350ebbf8')  
name: "Vijay"  
city: "Hyd"
```

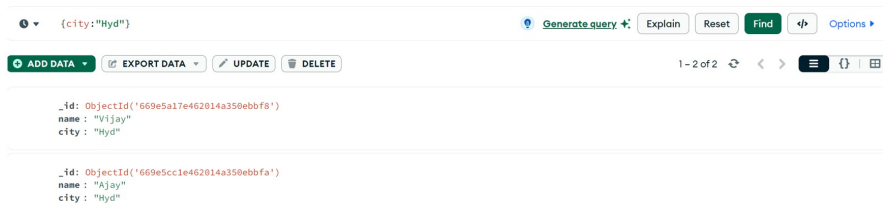
```
_id: ObjectId('669e5cc1e462014a350ebbf9')  
name: "Ajay"  
city: "Hyd"
```

```
_id: ObjectId('669e5cc1e462014a350ebbf9')  
name: "Shiva"  
city: "Mumbai"
```

We can update or delete the document by selecting the specific document

Using MongoDB Compass

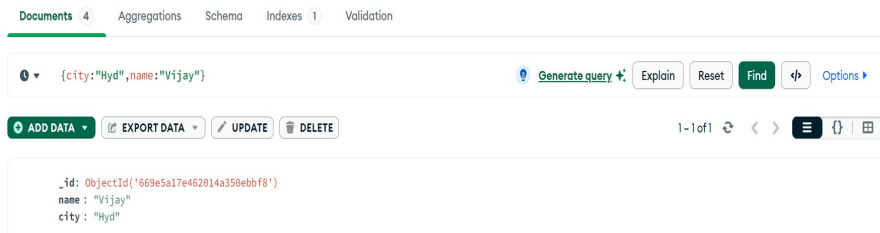
We can filter the documents by giving a query in the query box given above the collection as shown below:



In the above collection we are filtering the documents with city "Hyd"

Using MongoDB Compass

We can also give multiple queries to filter the documents



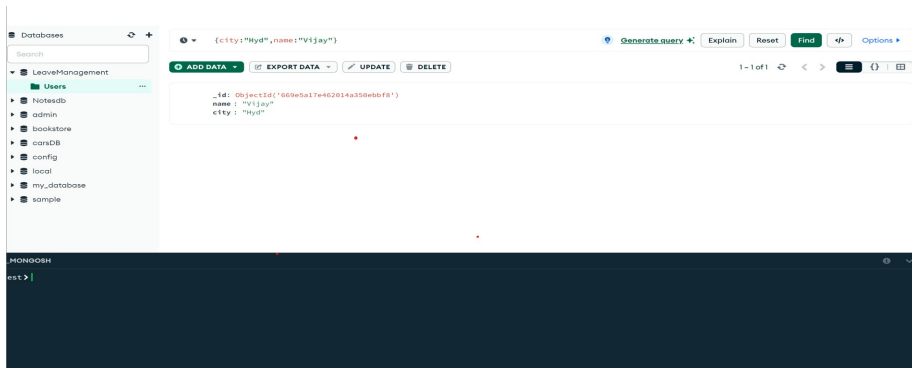
The screenshot shows the MongoDB Compass interface. At the top, there are tabs for Documents (4), Aggregations, Schema, Indexes (1), and Validation. The Documents tab is selected. Below the tabs, there is a query filter input field containing the JSON query `{city:"Hyd",name:"Vijay"}`. To the right of the filter are buttons for `Generate query`, `Explain`, `Reset`, `Find`, and `Options`. Below the filter, there are buttons for `ADD DATA`, `EXPORT DATA`, `UPDATE`, and `DELETE`. On the right side, there is a status bar showing `1 - 1 of 1` and navigation icons. Below the status bar, a document is displayed with the following fields: `_id`: `ObjectId('669e5a17e462014a350ebbf8')`, `name`: `"Vijay"`, and `city`: `"Hyd"`.

In the above collection we are filtering the documents with city "Hyd" and name "Vijay"

Using MongoDB Shell

We can use Mongo Shell in two ways

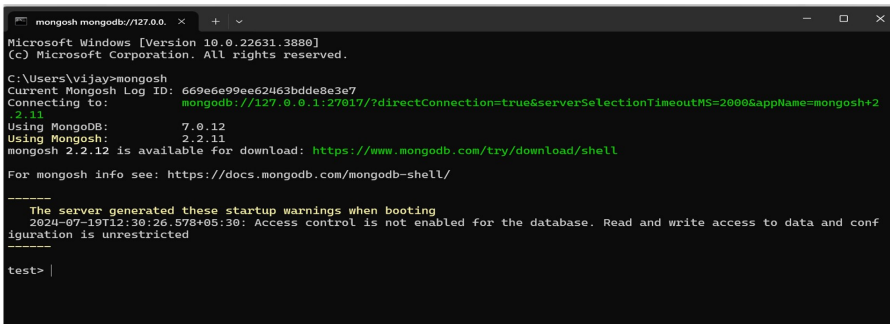
- We can directly use Mongosh from MongoDB Compass by selecting at the bottom as shown below:.



Using MongoDB Shell

We can also use command prompt to use mongo shell

- Open command prompt
- type mongosh



```
Microsoft Windows [Version 10.0.22631.3880]
(c) Microsoft Corporation. All rights reserved.

C:\Users\vijay>mongosh
Current Mongosh Log ID: 669e6e99ee62463bdde8e3e7
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.2.11
Using MongoDB:      7.0.12
Using Mongosh:       2.2.11
mongosh 2.2.12 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://docs.mongodb.com/mongodbd-shell/

-----
The server generated these startup warnings when booting
2024-07-19T12:30:26.578+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> |
```

Creating Database - Mongo Shell

- Open command prompt
- Navigate to mongo shell prompt by typing the command `mongosh`
- Use the following command either to switch or create a new database **use databasename**

Ex: use usermanagement

```
Current Mongosh Log ID: 669f1f03c481e7db7fe8e3e7
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.2.11
Using MongoDB:      7.0.12
Using Mongosh:       2.2.11
mongosh 2.2.12 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://docs.mongodb.com/mongosh-shell/

=====
  The server generated these startup warnings when booting
  2024-07-23T04:31:07.025+05:30: Access control is not enabled for the database. Read and write access to data and conf
  igation is unrestricted
=====

test> use usermanagement
switched to db usermanagement
usermanagement>
```

Note: Database and Collection will be created permanently by inserting at least one document.

Adding New Documents

Adding the documents to Collection can be done in two ways:

1. `insertOne()`: Insert one document into the collection

```
1 db.users.insertOne(  
2 {  
3   name:"Vijay",  
4   city:"Hyd",  
5   department:"CSE"  
6 }  
7 )  
8
```

```
usermanagement> db.users.insertOne({name:"Vijay",city:"Hyd",department:"CSE"})  
{  
  acknowledged: true,  
  insertedId: ObjectId('669f22ffc481e7db7fe8e3e9')  
}  
usermanagement> |
```

An ObjectId will be generated automatically after inserting a new document

Adding New Documents

2. insertMany(): Insert Multiple documents into the collection

```
1 db.users.insertMany(  
2 [  
3 {name:"Ajay",city:"Hyd",department:"CSE"},{name:"Shiva",  
city:"Wgl",department:"ECE"}  
4 ]  
5 )  
6
```

```
usermanagement> db.users.insertMany([{name:"Ajay",city:"Hyd",department:"CSE"},{name:"Shiva",city:"Wgl",department:"ECE"}])  
{  
  acknowledged: true,  
  insertedIds: {  
    '0': ObjectId('669f2638c481e7db7fe8e3ea'),  
    '1': ObjectId('669f2638c481e7db7fe8e3eb')  
  }  
}  
usermanagement> |
```

Finding Documents

We use find() method for finding the documents

```
1 db.users.find()  
2
```

This method gives us first 20 documents of the collection.
Note: If we want more no.of documents we need to type **it** at the mongosh to get more 20 documents like this it goes on...

```
1 db.users.find({department:"CSE"})  
2
```

Filters the data based on the single search criteria

```
1 db.users.find({department:"CSE",city:"Hyd"})  
2
```

Filters the data based on multiple search criteria

Finding Documents

```
1 4.      db.users.find({department:"CSE",city:"Hyd"},{name:1,  
2          department:1})
```

Retrieve the specific fields of the documents based on search criteria

```
1 5.      db.users.find({}, {name:1, department:1})  
2
```

Retrieve the specific fields of all the documents with empty search criteria

findOne(): we can also use findOne() method to get only one document from the search results

```
1      db.users.findOne({department:"CSE"})  
2
```

show collections: Display all the collections in the database

Sorting and Limiting Data

sort(): It is the method used to sort the documents ascending or descending order

Ascending order:

```
1 db.users.find().sort({name:1})  
2
```

Descending order:

```
1 db.users.find().sort({name:-1})  
2
```

count(): It is the method used to get the no. of documents from the collection

```
1 db.users.count()  
2
```

Sorting and Limiting Data

limit(): It is the method used to get the limited no.of documents from the collection.

```
1 db.users.find().limit(3)
```

We can also use chaining of methods as below:

```
1 db.users.find({department:"CSE"}).limit(2)
```

```
1 db.users.find({department:"CSE"}).count()
```

Nested Documents

In MongoDB we can embed/nest a document inside another document

Syntax:

```
1  {  
2  ....  
3  field: {field1: value1, field2: value2}  
4  ....  
5  }  
6
```

Example

```
1  db.users.insertOne(  
2  {  
3  name:{firstname:"Vijay",lastname:"kumar"},  
4  city:"Hyd",  
5  department:"CSE"  
6  }  
7  )  
8
```

Operators

- MongoDB provides us the operators for filtering the documents with complex queries
- Every operator in MongoDB to be prefixed with \$ sign
- **\$or:** It is an operator used to filter the document with multiple search criteria, it is similar to or condition in programming. languages

```
1 Ex: db.users.find({$or:[{name:"Vijay"},{city:"Hyd"}]})
```

```
2
```

- **\$in:** It is an operator used to filter the documents in between the multiple values for a single field.

```
1 Ex: db.users.find({city:{$in: ["Hyd","Wgl"]}})
```

```
2
```

- **\$nin:** It is an operator used to filter the documents not in between the multiple values for a single field.

```
1 Ex: db.users.find({city:{$nin: ["Hyd"]}})
```

```
2
```

Operators

- **\$gt:** It is an operator used to filter the documents with single field or multiple fields greater than the specified value.

```
1 Ex: db.users.find({age:{$gt: 32}})
```

```
2
```

- **\$gte:** It is an operator used to filter the documents with single field or multiple fields greater than or equal to the specified value.

```
1 Ex: db.users.find({age:{$gte: 32}})
```

```
2
```

- **\$lt:** It is an operator used to filter the documents with single field or multiple fields lesser than the specified value.

```
1 Ex: db.users.find({age:{$lt: 32}})
```

```
2
```

- **\$lte:** It is an operator used to filter the documents with single field or multiple fields lesser than or equal to the specified value.

```
1 Ex: db.users.find({age:{$lte: 32}})
```

```
2
```

Updating Documents

Updating the documents to Collection can be done in two ways:

1. `updateOne()`: It is the method used to update one document in the collection

- **\$set**: It is an operator used to update the specific field or multiple field values based on the filter criteria

```
1
2 db.users.updateOne({_id:ObjectId('668
   e55bbd4714999ffe8e3e9')},{ $set:{name:"Vijay Kumar",
   age:35}})
3
```

Updating Documents

- **\$inc:** It is an operator used to increment the field value by the given number.

```
1 db.users.updateOne({_id:ObjectId('668
    e55bbd4714999ffe8e3e9')},{ $inc:{age:5}})
2
```

- **\$push:** It is an operator used to push an element into the array of the field in the document of a collection.

```
1 db.users.updateOne({_id:ObjectId('668
    e55bbd4714999ffe8e3e9')},{ $push:{reviews:{title:"
    review3",body:"excellent"}}})
2
```

- **\$pull:** It is an operator used to pull an element from an array field in the document of a collection.

```
1 db.users.updateOne({_id: ObjectId('669
    f42d7c481e7db7fe8e3ee')},{ $pull:{hobbies:"h3"}})
2
```

Updating Documents

1. `updateMany()`: It is the method used to update the multiple documents based on the search criteria.

- **\$set:**

```
1 db.users.updateMany({name:"shiva"},{$set: {name:"Shiva"  
    }})  
2  
3 db.users.updateMany({name:"Shiva"},{$set: {name:"  
    s h i v a ,age:25}})  
4
```

- **\$inc:**

```
1 db.users.updateMany({name:"shiva"},{$inc: {age:2}})  
2
```

- **\$push:**

```
1 db.users.updateMany({name:"shiva"},{$push: {hobbies:  
    reading    }})  
2
```

Updating Documents

- **\$pull:**

```
1 db.users.updateMany({name:"shiva"},{$pull: {hobbies:  
      reading      }})
```

2

Deleting Documents

Deleting the documents to Collection can be done in two ways:

1. `deleteOne()`: This method is used to delete one document from a collection.

```
1 db.users.deleteOne({_id: ObjectId('668  
    f62557442aa754eeb7f83')})
```

1. `deleteMany()`: This method is used to delete one or more documents from a collection.

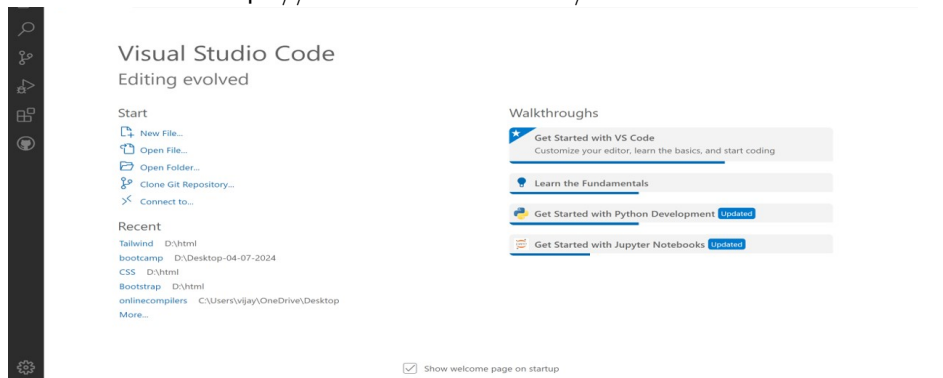
```
1 db.users.deleteMany({age:32})  
2  
3 db.users.deleteMany({age:32,name:    vijay    })
```

Connecting to MongoDB using Nodejs

Step 1:

Install Visual code editor using following link

Download Link: <https://code.visualstudio.com/>



Connecting MongoDB using Nodejs

Step 2:

We need to install Node.js and npm(Node package manager)

Download Link: <https://nodejs.org/en/download/prebuilt-installer>

Download Node.js®

Download Node.js the way you want.

Package Manager **Prebuilt Installer** Prebuilt Binaries Source Code

I want the **v20.15.1 (LTS)** version of Node.js for **Windows** running **x64**

 **Download Node.js v20.15.1**

Node.js includes npm (10.7.0) ↗.

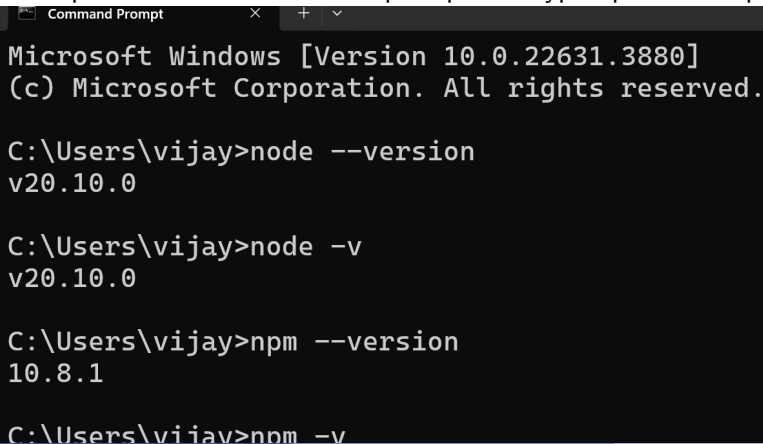
[Read the changelog for this version](#) ↗

Connecting to MongoDB using Nodejs

Step 3:

Verify Installation

- Node: Go to the command prompt and type `node -v` or `node --version`
- npm: Go to the command prompt and type `npm -v` or `npm --version`



```
Microsoft Windows [Version 10.0.22631.3880]
(c) Microsoft Corporation. All rights reserved.

C:\Users\vijay>node --version
v20.10.0

C:\Users\vijay>node -v
v20.10.0

C:\Users\vijay>npm --version
10.8.1

C:\Users\vijay>npm -v
```

Connecting to MongoDB using Nodejs

Mongoose: Inorder to connect to MongoDB using Nodejs we need to install mongoose

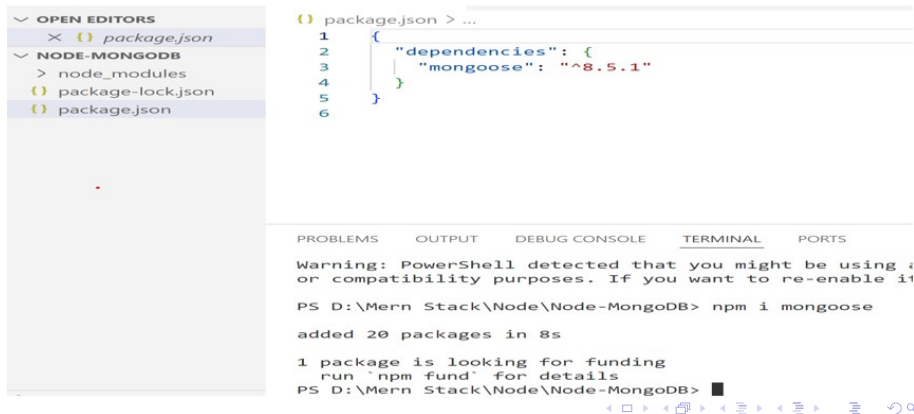
- Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js.
- It is used to give schema-based solution to model your application data.
- Mongoose, you can define schemas for your documents, which are then translated into MongoDB collections.

Connecting to MongoDB using Nodejs

Install Mongoose:

Open visual studio code terminal and type the following command

```
npm i mongoose
```



```
{  
  "dependencies": {  
    "mongoose": "^8.5.1"  
  }  
}
```

```
Warning: PowerShell detected that you might be using it  
or compatibility purposes. If you want to re-enable it  
  
PS D:\Mern Stack\Node\Node-MongoDB> npm i mongoose  
  
added 20 packages in 8s  
  
1 package is looking for funding  
  run `npm fund` for details  
PS D:\Mern Stack\Node\Node-MongoDB>
```

Connecting to MongoDB using Nodejs

Steps to connect to MongoDB:

- Create a project folder in the machine.
- Open project folder using Visual studio Code.
- Create a new file called app.js
- Copy the database URL from mongoDB compass installed in the machine.
- Write the following code to connect to MongoDB

```
1
2 const mongoose=require("mongoose");
3 mongoose.connect("mongodb://localhost:27017/
  usermanagement").then( ()=>{
4   console.log("connected");
5 });
6
```

- Run the project using the following command
`node app`

Connecting to MongoDB using Nodejs

Check the below screen shot for the reference:

The screenshot shows the VS Code interface with the Explorer, Open Editors, and Node-MongoDB panels. The Explorer panel shows the file structure with 'app.js' selected. The Open Editors panel shows 'app.js' open. The Node-MongoDB panel shows the connection status as 'connected'. The terminal panel shows the command 'node app' executed, resulting in the output 'connected'.

```
EXPLORER
...
JS app.js X

OPEN EDITORS
X JS app.js

NODE-MONGODB
> node_modules
JS app.js
{} package-lock.json
{} package.json

JS app.js > ...
1  const mongoose=require("mongoose");
2  mongoose.connect("mongodb://localhost:27017/usermanagement").then(()=>{
3      console.log("connected");
4  });

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\Mern Stack\Node\Node-MongoDB> node app
connected
```

:

- Database is usermanagement
- If we connect to MongoDB successfully we get "connected" message at the console.

Connecting to MongoDB using Nodejs

Check the below screen shot for the reference:

The screenshot shows the VS Code interface. On the left, the Explorer sidebar shows the file structure with 'app.js' selected. The main editor displays the content of 'app.js', which contains the following code:

```
1 const mongoose=require("mongoose");
2 mongoose.connect("mongodb://localhost:27017/usermanagement").then(()=>{
3   console.log("connected");
4 });
```

Below the editor, the TERMINAL panel is active, showing the command prompt output:

```
PS D:\Mern Stack\Node\Node-MongoDB> node app
connected
```

:

- Database is usermanagement
- If we connect to MongoDB successfully we get "connected" message at the console.

Reading Data from MongoDB

In order to read or write the data into the collection we need to create Schema and Model of the collection in the database

Example: users collection

```
1  const Schema = mongoose.Schema;
2  const userSchema = new Schema({
3      name: String,
4      city: String,
5      department: String
6  });
7  const User = mongoose.model('User', userSchema);
8
```

Reading Data from MongoDB

Here is the sample code to read the data from the users collection of usermanagement database
Create a new file readdata.js and write the following code in it

```
1 const mongoose = require('mongoose');  
2 const mongoURI = 'mongodb://localhost:27017/usermanagement';  
   // Replace with your connection string  
3 mongoose.connect(mongoURI)  
4 .then(() => {  
5     console.log('MongoDB connected successfully.');6     const Schema = mongoose.Schema;  
7     const userSchema = new Schema({  
8         name: String,  
9         city: String,  
10        department: String  
11    });  
12
```

Reading Data from MongoDB

Sample Code continued...

```
1 const User = mongoose.model('users', userSchema);
2   User.find()
3     .then(users => {
4       console.log('Users:', users);
5     })
6     .catch(err => {
7       console.error('Error fetching users:', err);
8     });
9
10 })
11 .catch((err) => {
12   console.error('MongoDB connection error:', err);
13 });
14
```

Execute the project using the following command
node readdata

Read Data from MongoDB

output:

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS D:\Mern Stack\Node\Node-MongoDB> node readdata
MongoDB connected successfully.
Here's is the users data
Users: [
  {
    _id: new ObjectId('669f22ffc481e7db7fe8e3e9'),
    name: 'Vijay',
    city: 'Hyd',
    department: 'CSE'
  },
  {
    _id: new ObjectId('669f2638c481e7db7fe8e3ea'),
    name: 'Ajay',
    city: 'Hyd',
  }
]
```

Writing Data to MongoDB

In order to read or write the data into the collection we need to create Schema and Model of the collection in the database

Example: users collection

```
1  const Schema = mongoose.Schema;
2  const userSchema = new Schema({
3      name: String,
4      city: String,
5      department: String
6  });
7  const User = mongoose.model('User', userSchema);
8
```

Writing Data to MongoDB

Here is the sample code to write the data into the users collection of usermanagement database

Create a new file writedata.js and write the following code in it

```
1 const mongoose = require('mongoose');
2 const mongoURI = 'mongodb://localhost:27017/usermanagement';
   // Replace with your connection string
3 mongoose.connect(mongoURI)
4 .then(() => {
5     console.log('MongoDB connected successfully.');
```

// Define a schema

```
7 const Schema = mongoose.Schema;
8 const userSchema = new Schema({
9     name: String,
10    city: String,
11    department:String
12 });
```

Writing Data to MongoDB

Sample Code continued...

```
1 // Create a model
2 const userModel = mongoose.model('users', userSchema);
3 // Use the model to create a document
4 const newUser = new userModel({
5   name: 'Ramu',
6   city: "Hyd",
7   department: "ECE"
8 });
9 // Save the document to the database
10 newUser.save()
11 .then(doc => {
12   console.log('Document saved:', doc);
13 })
14 .catch(err => {
15   console.error('Error saving document:', err);
16 });
17 })
18 .catch((err) => {
```

Writing Data to MongoDB

Sample Code continued...

```
1 .catch((err) => {  
2     console.error('MongoDB connection error:', err);  
3 });  
4
```

Execute the project using the following command
node writedata

output:

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS D:\Mern Stack\Node\Node-MongoDB> node writedata  
MongoDB connected successfully.  
Document saved successfully: {  
  name: 'Ramu',  
  city: 'Hyd',  
  department: 'ECE',  
  _id: new ObjectId('66a1e858c0933a638a942f86'),  
  __v: 0  
}
```

Writing Data to MongoDB

If we observe in the output screenshot there is a new field

```
1  __v:0
```

is added automatically by mongoose while adding a new document into the collection.

- This key is used by Mongoose to track the version of each document.
- We can exclude this by giving versionKey: false while creating the mongoose schema as shown below:

```
1  const Schema = mongoose.Schema;  
2  const userSchema = new Schema({  
3    name: String,  
4    city: String,  
5    department:String  
6  }, {versionKey: false});  
7
```

Sample Code

Github Repository

<https://github.com/bolluvijaykumar/nodejs-mongodb.git>

Steps: To run the sample code

- 1 Download the zip file from the repository
- 2 Extract and open the firstproject folder using VS Code
- 3 Open terminal and execute the following command to install the node and mongoose
`npm i mongoose`
- 4 Run the command "node app" to test connecting to mongodb database
- 5 Run the command "node writedata" to test writing data to the users collection
- 6 Run the command "node readdata" to test reading data from the users collection

Questionnaire

Small Answer Questions

- 1 How does MongoDB store data?
- 2 How are collections and documents related in MongoDB?
- 3 What features does MongoDB Compass provide for managing databases?
- 4 What command is used to show all databases in MongoDB Shell?
- 5 How do you switch to a specific database in MongoDB Shell?

Long Answer Questions

- 1 Explain the key features and benefits of using MongoDB as a NoSQL database compared to traditional relational databases.
- 2 Describe the relationship between databases, collections, and documents in MongoDB. How does this hierarchy differ from the table-based structure of relational databases?
- 3 Describe the key features of MongoDB Compass. How can Compass be used to perform operations like creating collections, inserting documents, querying data, and visualizing schema?
- 4 Describe the steps involved in setting up a connection to MongoDB from a Node.js application. What packages are commonly used, and how are they configured?
- 5 Describe the process of reading data from a MongoDB database using Mongoose in a Node.js application. What methods are available, and how are they used?

A set of navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.